

Doc. No.: WG21/P0270R0

Date: 2016-02-12

Author: Hans-J. Boehm

Email: hboehm@google.com

P0270R0: Removing C dependencies from signal handler wording

This topic was briefly addressed on P0063R0, but the authors subsequently decided to address it separately.

The current C++ working draft restricts signal handlers to functions in the intersection of C and C++. We believe this is dubious for at least three reasons:

- The semantics implicitly change if we refer to a newer version of the C standard. P0063R1 proposes to normatively refer to C11 instead of C99. This, for example, raises the question of whether `thread_local` variables, which are supported in C11 with slightly different keyword conventions, are now allowed in signal handlers. This was never our intent, since it would preclude some implementation techniques.
- The current wording is not as clear as it should be. For example, Richard Smith points out that C functions, and hence POFs cannot have C linkage, since there is no way to specify C linkage in C.
- The current restrictions don't appear to reflect those that are really necessary or desirable in signal handlers. There is no good reason that signal handlers can't mention `nullptr` for example. The real restrictions we need are that such functions should have C linkage, should not introduce data races (already addressed by the "potentially concurrent" notion in 1.10), and if they avoid certain facilities that are frequently implemented in a signal-unsafe manner, particularly anything that may require a hidden lock.

We propose to remedy this by replacing the notion of a "POF", "plain old function" with a "signal-safe function" defined entirely in C++ terms. The definition we use largely excludes use of the library, thus avoiding any need to reimplement that in a signal-safe manner. This makes the definition rather restrictive. But we believe it remains at least theoretically possible for the user to write very sophisticated signal handlers through the use of atomics.

We expect that, although these changes are clearly substantive, there is no real practical impact. Existing implementations and user code should continue to work unchanged (or remain as broken as they always were.)

We do not exclude lambda expressions from signal handlers, on the assumption that the lambda expression itself should not require memory allocation. We do not say anything about non-function-local statics, since it should not be possible to trigger construction or destruction from within a signal handler.

We somewhat separately propose to make the behavior of non-signal-safe functions in signal handlers undefined, instead of implementation-defined. We are not aware of any implementations that actually attempt to define this.

We do not exclude lambda expressions from signal handlers, on the assumption that the lambda expression itself should not require memory allocation.

The restriction on static initialization warrants discussion. We currently allow even the call of a POF that doesn't access any globals to trigger static initialization (3.6.2p4 [basic.start.init]). It's not clear how to make that work if a thread already part way through the initialization receives the signal. This appears to me to be a preexisting problem that this exposes, rather than a problem introduced by this change.

It may make sense to eventually relax these restrictions invoked through an explicit synchronous `raise()` call. It's unclear that this is important enough to bother.

These changes are arguably not essential for the adoption of the P0063. But we believe they are highly desirable and should follow P0063 in fairly short order.

Proposed wording changes:

Update 18.10p10 [support.runtime] as follows:

~~The common subset of the C and C++ languages consists of all declarations, definitions, and expressions that may appear in a well formed C++ program and also in a conforming C program. A POF ("plain old function")~~ signal-safe function ~~is a function that uses only features from this common subset, and that does not directly or indirectly use any function that is not a POF, has C linkage, and neither directly nor indirectly:~~

- Calls any library functions defined in clause 18 through 30, except that it may call a function explicitly identified as signal-safe, and except that it may use operations on `nullptr_t` and plain lock-free atomic operations. A plain lock-free atomic operation is an invocation of a function `f` from Clause 29, such that `f` is not a member function, and either `f` is the function `atomic_is_lock_free()`, or the member function is `lock_free()`, or for every atomic argument `A` passed to `f`, and every atomic object of class `A` on which a member function is invoked, `atomic_is_lock_free(A)` yields true. All signal handlers shall have C linkage. [Note: This implicitly excludes the use of new and delete expressions that rely on a library-provided memory allocator. -- end note]
- Accesses a thread duration object.
- Throws an exception.
- Initializes a variable of static duration and the initialization is not a static initialization.

The behavior of any function other than a ~~POF~~signal-safe function used as a signal handler in a C++ program is ~~implementation-defined~~ undefined.

The `signal()` function is signal-safe.

We suggest the following additions, partially as examples. We expect that a few other functions should also eventually be declared signal-safe, but these are among the more blatant cases. `abort()` and `quick_exit()` are treated as safe by C.

Add after the title of 18.3.2.4 [numeric.limits.members]:

Each member function defined in this section is signal-safe (18.10 [support.runtime]).

Add at the end of 18.5p3 [support.start.term]:

The `Exit()` function is signal-safe (18.10 [support.runtime]).

Add at the end of 18.5p4 [support.start.term]:

The `abort()` function is signal-safe (18.10 [support.runtime]).

Add at the end of 18.5p10 [support.start.term]:

The `quick_exit()` function is signal-safe (18.10 [support.runtime]). [Note: It may still be unsafe to call from a handler, because the functions registered with `at_quick_exit()` may not be. --end note]

Acknowledgment

Clark Nelson provided many useful suggestions, but not necessarily agreement.